Reinventing Primality Testing through Randomized Algorithms

Aaryan Maheshwari(230022) Md Arsal Asif (230643) Aruz Awasthi(230209) Suyash Kapoor (231066)

April 5, 2026

Abstract

Primality testing is a fundamental problem in number theory and computer science, with applications in cryptography. This report studies randomized algorithms for primality testing, including Fermat's test and the Miller–Rabin test. We analyze their correctness, runtime, and probability of error, and present experimental results comparing their performance.

Contents

1 Introduction **3**

2 Deterministic Algorithms **3**
2.1 $O(\sqrt{N})$-time Algorithm . . . 3
2.2 Sieve of Eratosthenes . . . 4
2.3 AKS Primality Test . . . 4

3 A Randomized Algorithm **4**
3.1 Introduction . . . 4
3.2 Fermat's Little Theorem . . . 4
3.3 Meeting 1: Experimental Investigation of Fermat's Condition . . . 5
3.4 Meeting 2: Partitioning A and the Membership of A_{fermat} . . . 5
3.5 Identification of the "Outliers" in A_{fermat} Density . . . 6
3.6 Proof of the "Blind Spot" in Carmichael Numbers . . . 6
3.7 Meeting 3: The Half-Density Proof ($|A_{fermat}| \leq \frac{|A'|}{2}$) . . . 7
3.8 Final Randomized Algorithm . . . 7
3.9 Time Complexity Analysis . . . 8
3.10 Error Analysis . . . 8
3.10.1 Case 1: n is Composite but not Carmichael . . . 8
3.10.2 Case 2: n is a Carmichael Number . . . 9

4 Another Randomized Algorithm **9**
4.1 The Idea . . . 9
4.2 The Sequence of Repeated Squaring . . . 10
4.3 Final Randomized Algorithm . . . 10
4.4 Error Analysis . . . 11
4.5 Time Complexity Analysis . . . 11
4.6 Comparison and Reliability . . . 11

5 Experimentation and Benchmarking **12**
5.1 AKS Implementation 12
5.2 Miller-Rabin Implementation 12
5.3 Analysis 13
5.4 Deviation from Average Runtime 14

6 Interactive Exploration and Analysis of Miller-Rabin vs. AKS **16**
6.1 Asymptotic vs. Empirical Complexity 16
6.2 Probabilistic Error Bounds and Hardware Realities 17
6.3 Practical Application: RSA Key Generation 17

7 Conclusion **18**

8 Acknowledgment **19**

1 Introduction

Primality testing is a fundamental problem in **number theory** and **computer science**, with critical applications in **cryptography**, including **RSA** and **Diffie–Hellman key exchange**, where large prime numbers are essential. While the breakthrough deterministic polynomial-time **AKS algorithm**, developed at IIT Kanpur, established that primality testing lies in **P**, its practical implementation remains computationally intensive. In contrast, **randomized algorithms** continue to dominate in practice due to their simplicity, efficiency, and strong **probabilistic guarantees**.

In this project, we study **randomized Monte Carlo methods** for primality testing, focusing on the **Miller–Rabin algorithm**. Built upon **modular arithmetic** and **Fermat's Little Theorem**, these methods achieve significantly faster performance, with time complexity $O(k \log^2 n)$, where k controls the trade-off between runtime and accuracy. We analyze the probabilistic structure of these algorithms, including their error behavior and the role of exceptional composite numbers such as **Carmichael numbers**.

We implement the **Miller Rabin test** from first principles and evaluate its performance across inputs of varying sizes, comparing it with naive deterministic approaches. Our results demonstrate that with a modest number of iterations, the probability of error decreases exponentially, rendering the algorithm effectively deterministic in practice. This study highlights the practical superiority of **randomized algorithms** for large-scale primality testing and their indispensable role in modern computational and cryptographic applications.

2 Deterministic Algorithms

2.1 $O(\sqrt{N})$-time Algorithm

By definition of primes, a number N is prime if it has no positive divisors other than 1 and itself. This leads to an algorithm that checks whether any integer d in the range $[2, \lfloor\sqrt{N}\rfloor]$ divides N. If any such divisor is found, N is composite; otherwise, it is prime.

Algorithm 1: Primality Testing (Trial Division)

```
function Check_Primality(N)
    for i = 2 to ⌊√N⌋ do
        if N mod i = 0 then
            return Composite
        end if
    end for
    return Prime
end function
```

In the Word RAM model, this algorithm takes

$$O(\sqrt{N}) = O\left(2^{n/2}\right)$$

time to test the primality of N, where $n = \log N$ is the number of bits.

2.2 Sieve of Eratosthenes

The Sieve of Eratosthenes is an efficient deterministic algorithm for finding all prime numbers up to a given limit N. It works by iteratively marking the multiples of each prime starting from 2. Numbers that remain unmarked are prime.

Key Idea:

- Start with all numbers from 2 to N
- Repeatedly mark multiples of each prime
- The remaining unmarked numbers are primes

The time complexity of the sieve is

$$O(N \log \log N)$$

which is significantly faster than checking each number individually for primality when generating primes in bulk.

2.3 AKS Primality Test

The AKS primality test was introduced in 2002 at IIT Kanpur. It was the first deterministic primality testing algorithm proven to run in polynomial time in the size of the input.

Its time complexity is

$$O(n^6) = O((\log N)^6)$$

for an integer N, where $n = \log N$ is the number of bits.

Despite its theoretical importance, the AKS algorithm is not practical for large inputs due to its high computational cost compared to randomized algorithms.

3 A Randomized Algorithm

3.1 Introduction

The initial idea for our randomized primality testing algorithm focused on exploring the applicability of Fermat's Little Theorem. Deterministic algorithms, while guaranteeing absolute correctness, often suffer from high computational complexity, making them impractical for extremely large inputs. Randomized algorithms offer a powerful alternative by providing significantly faster runtimes with a boundable margin of error. The theoretical foundation of this approach relies on a fundamental property of prime numbers established by Pierre de Fermat.

3.2 Fermat's Little Theorem

Theorem: For any integer a and a prime number p, the following property always holds:

$$a^p \equiv a \pmod{p}$$

If a is not divisible by p (that is, if $\gcd(a, p) = 1$), Fermat's Little Theorem can be written in the following equivalent, and highly useful, form:

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof:
Let p be a prime number and a be an integer such that $\gcd(a, p) = 1$. Consider the set of the first $p - 1$ positive multiples of a:

$$S = \{a, 2a, 3a, \ldots, (p-1)a\}$$

First, we show that no two elements in this set are congruent modulo p. Suppose, for the sake of contradiction, that $i \cdot a \equiv j \cdot a \pmod{p}$ for some $1 \leq i < j \leq p - 1$. This implies that $p \mid a(j - i)$. Since p is prime and $\gcd(a, p) = 1$, by Euclid's lemma, p must divide $j - i$. However, $0 < j - i < p$, making this impossible. Thus, all elements in S are distinct modulo p.

Since there are $p - 1$ distinct elements in S and none are congruent to $0 \pmod{p}$ (as p does not divide a or any multiplier less than p), the set S taken modulo p must simply be a permutation of the set of integers $\{1, 2, \ldots, p - 1\}$.

Therefore, the product of all elements in S must be congruent to the product of the first $p - 1$ integers modulo p:

$$a \cdot 2a \cdot 3a \cdots (p-1)a \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}$$

$$a^{p-1} \cdot (p-1)! \equiv (p-1)! \pmod{p}$$

Since p is prime, it does not divide any of the factors making up $(p-1)!$. This means $\gcd((p-1)!, p) = 1$, allowing us to cancel $(p - 1)!$ from both sides of the congruence. This yields:

$$a^{p-1} \equiv 1 \pmod{p}$$

■

If a number n is prime, this condition must hold true for all $1 \leq a < n$. Therefore, if we randomly select a base a and observe that $a^{n-1} \not\equiv 1 \pmod{n}$, we can definitively declare n as composite. The following sections detail our theoretical and empirical investigation into utilizing this theorem for primality testing.

3.3 Meeting 1: Experimental Investigation of Fermat's Condition

The objective was to empirically evaluate the reliability of the condition $a^{n-1} \equiv 1 \pmod{n}$ for identifying composite numbers. We define the set of bases satisfying this condition as:

$$A_{fermat} = \{a \in \mathbb{Z} \mid 1 \leq a < n \text{ and } a^{n-1} \equiv 1 \pmod{n}\}$$

By analyzing composite n up to 10,000, we observed that for most n, the ratio $|A_{fermat}|/(n-1)$ is extremely low. However, we identified "outlier" cases (Carmichael numbers) where this ratio remains high, necessitating a deeper theoretical partition of the search space.

3.4 Meeting 2: Partitioning A and the Membership of A_{fermat}

We partition the set of all residues $A = \{a \mid 1 \leq a < n\}$ into two disjoint sets:

- $A' = \{a \in A \mid \gcd(a, n) = 1\}$ (Units modulo n)
- $A'' = \{a \in A \mid \gcd(a, n) > 1\}$ (Non-units)

Theorem 1: $A_{fermat} \subseteq A'$.
Proof: Let $a \in A_{fermat}$. Then $a^{n-1} \equiv 1 \pmod{n}$. This implies there exists an integer k such that $a^{n-1} - kn = 1$, or $a(a^{n-2}) - kn = 1$. By Bézout's Identity, $\gcd(a, n)$ must divide any linear combination of a and n. Since $\gcd(a, n) \mid 1$, it follows that $\gcd(a, n) = 1$. Thus, $a \in A'$.

Conclusion: No element of A'' can belong to A_{fermat}. Therefore, discovering an element a where $\gcd(a, n) > 1$ provides immediate proof that n is composite.

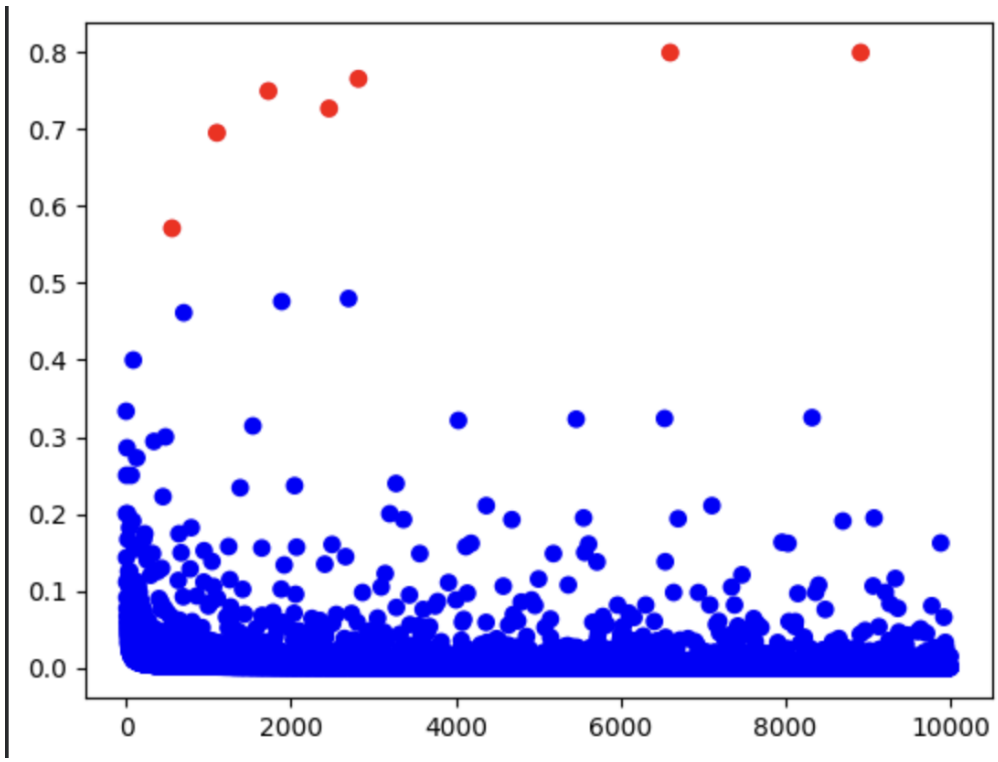

Figure 1: Distribution of the density of A_{fermat} for composite n. Red points indicate Carmichael numbers where the density exceeds 0.5.

3.5 Identification of the "Outliers" in A_{fermat} Density

Our experimental data for $n < 10,000$ (see Figure 1) demonstrates that for the vast majority of composite numbers, the density of A_{fermat} is well below 0.5. However, we observe a small set of anomalous composite numbers where the ratio $\frac{|A_{fermat}|}{n-1}$ is significantly higher, appearing to approach the total count of residues coprime to n.

By examining these specific values (e.g., $561, 1105, 1729$), we note a peculiar property: for these numbers, the Fermat condition $a^{n-1} \equiv 1 \pmod{n}$ holds for **every** $a \in A'$ (see Figure 2. We formally define such numbers based on this observed property.

Definition: A composite number n is a *Carmichael number* if:

$$A' \subseteq A_{fermat}$$

This implies that for a Carmichael number, every element that is coprime to n satisfies the Fermat condition, mimicking the behavior of a prime number.

3.6 Proof of the "Blind Spot" in Carmichael Numbers

We must prove that if n is a Carmichael number, no element in A' can reveal n as composite through the Fermat condition.

Theorem 3: If n is a Carmichael number, then $A' \setminus A_{fermat} = \emptyset$.

Proof: By the definition established in the previous section, n is Carmichael if and only if for all a such that $\gcd(a, n) = 1$, the congruence $a^{n-1} \equiv 1 \pmod{n}$ holds. The statement $A' \subseteq A_{fermat}$ is equivalent to saying that every element of A' is also an element of A_{fermat}. Consequently, the set

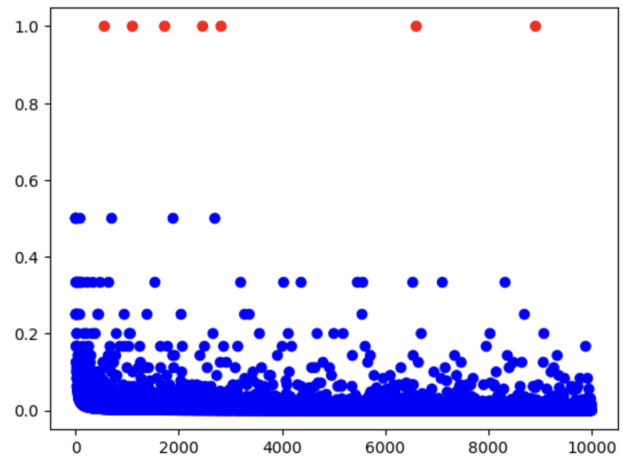

Figure 2: Distribution of the density of A_{fermat} taken against the set A' for composite n. Red points indicate Carmichael numbers where the density exceeds 0.5.

difference $A' \setminus A_{fermat}$ (the set of elements in A' that do *not* satisfy the Fermat condition) is the empty set.

Impact on Detection: This means that for Carmichael numbers, the only elements that can reveal n as composite are those in A'' (elements where $\gcd(a, n) > 1$). For large n, the probability of randomly picking an element from A'' is typically very low, as $|A''| = (n-1) - \phi(n)$. ($\phi(n)$ is the Euler's-Totient function which denotes the number of elements in the set A').

3.7 Meeting 3: The Half-Density Proof ($|A_{fermat}| \leq \frac{|A'|}{2}$)

For any composite n that is not a Carmichael number, there exists at least one element $x \in A'$ such that $x \notin A_{fermat}$. We use this to bound the size of A_{fermat}.

Let $L = A_{fermat}$ (which we know is a subset of A'). Consider the set $xL = \{(x \cdot a) \pmod{n} \mid a \in L\}$.

1. **Injectivity:** If $x \cdot a_i \equiv x \cdot a_j \pmod{n}$, then $a_i \equiv a_j$ because $x \in A'$ is invertible. Thus, $|xL| = |L|$.

2. **Exclusion:** For any $a \in L$, $(x \cdot a)^{n-1} \equiv x^{n-1} \cdot a^{n-1} \equiv x^{n-1} \cdot 1 \not\equiv 1 \pmod{n}$.

This proves that $xL \cap A_{fermat} = \emptyset$. Since both L and xL are disjoint subsets of A', their combined size cannot exceed $|A'|$. Thus, $|L| + |xL| \leq |A'|$, which simplifies to:

$$|A_{fermat}| \leq \frac{|A'|}{2}$$

3.8 Final Randomized Algorithm

Synthesizing the above proofs, we define a randomized procedure that attempts to find an element in $A \setminus A_{fermat}$.

```
Algorithm: Fermat_Primality_Test(n, k)
-----------------------------------------
1. If n is even, return COMPOSITE.
2. Repeat k times:
    a. Pick a random 'a' in the range [2, n-2].
    b. Compute g = gcd(a, n).
    c. If g > 1:
        Return COMPOSITE (Found an element in A'').
    d. Compute r = a^(n-1) mod n.
    e. If r != 1:
        Return COMPOSITE (Found an element in A' \ A_fermat).
3. Return PRIME.
```

3.9 Time Complexity Analysis

The total time complexity of the Fermat Primality Test is $\mathcal{O}(k \cdot \log^3 n)$. The complexity is broken down as follows for each of the k iterations:

- **Greatest Common Divisor:** Computing $\gcd(a, n)$ via the Euclidean Algorithm requires $\mathcal{O}(\log^2 n)$ bit operations. This step is computationally "cheaper" than the subsequent exponentiation.
- **Modular Exponentiation:** Calculating $a^{n-1} \pmod{n}$ is the bottleneck of the algorithm. Utilizing the binary exponentiation (square-and-multiply) technique, the process requires $\mathcal{O}(\log n)$ modular multiplications.
- **Multiplication Complexity:** Each modular multiplication involves numbers of size $\log n$ bits. Using standard multiplication, this takes $\mathcal{O}(\log^2 n)$ time.
- **Total Per Iteration:** Consequently, the time complexity per iteration is $\mathcal{O}(\log n \cdot \log^2 n) = \mathcal{O}(\log^3 n)$.

Across k iterations, the total complexity is $\mathcal{O}(k \cdot \log^3 n)$. If advanced multiplication algorithms are employed, the complexity can be reduced to approximately $\mathcal{O}(k \log^2 n)$.

3.10 Error Analysis

The reliability of our algorithm depends on the probability of reaching a "Probably Prime" conclusion for a composite input.

3.10.1 Case 1: n is Composite but not Carmichael

From Theorem 2 (Meeting 3), we know that for a non-Carmichael composite n:

$$|A_{fermat} \cap A'| \leq \frac{|A'|}{2}$$

When we select a random $a \in \{2, \ldots, n-2\}$:

1. If $a \in A''$, the algorithm returns "COMPOSITE" (Step 2c).
2. If $a \in A' \setminus A_{fermat}$, the algorithm returns "COMPOSITE" (Step 2e).

3. The algorithm only fails if we pick $a \in A_{fermat}$.

Since $A_{fermat} \subseteq A'$, the probability P of failing to detect compositeness in a single iteration is:

$$P(\text{fail}) = \frac{|A_{fermat}|}{n-3} \leq \frac{|A'|/2}{n-3}$$

Because $n-3$ is approximately n and $|A'|$ is at most $n-1$, we can conservatively bound the error for one iteration as $P \leq \frac{1}{2}$. After k independent iterations, the probability that the algorithm incorrectly returns "PRIME" is:

$$P(\text{Error}) \leq \left(\frac{1}{2}\right)^k$$

For $k = 40$, the error probability is less than 10^{-12}, which is sufficient for most cryptographic applications.

3.10.2 Case 2: n is a Carmichael Number

As proven in Theorem 3, for these numbers, the Fermat check (Step 2e) will *never* reveal compositeness because $A' \setminus A_{fermat}$ is empty. Detection relies entirely on Step 2c ($\gcd(a, n) > 1$). The error probability in this case is:

$$P(\text{Error}) = \left(\frac{|A'| - 2}{n-3}\right)^k$$

Since $|A'|$ is often very close to n, this probability is nearly 1, demonstrating that the Fermat test is fundamentally non-rigorous for this specific class of outliers.

4 Another Randomized Algorithm

4.1 The Idea

Building on the insights from our earlier meetings, we sought to overcome the fundamental vulnerability of Fermat's Primality Test: the existence of Carmichael numbers. Our goal was to derive a stronger, foolproof condition for primality by leveraging the structural properties of numbers that we had already begun exploring. The critical breakthrough came from the following lemma:

Lemma: *If n is an odd prime number and $x^2 \equiv 1 \pmod{n}$, then $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$.*

Proof:
Let $x \equiv k \pmod{n}$, which can be written as:

$$x = \mu n + k, \quad \text{for some } \mu \in \mathbb{Z}$$

Squaring both sides gives:

$$x^2 = \mu^2 n^2 + 2\mu n k + k^2$$

Taking the equation modulo n, the terms with n drop out:

$$x^2 \equiv k^2 \pmod{n}$$

We are given that $x^2 \equiv 1 \pmod{n}$, so substituting this yields:

$$k^2 \equiv 1 \pmod{n}$$

Which implies:

$$k^2 - 1 \equiv 0 \pmod{n}$$

Factoring the difference of squares, we obtain:

$$(k-1)(k+1) \equiv 0 \pmod{n}$$

This congruency means that n divides the product $(k-1)(k+1)$. In equation form:

$$(k-1)(k+1) = \gamma n, \quad \text{for some } \gamma \in \mathbb{Z}$$

Now, since n is a prime number dividing a product, it must divide at least one of the factors. Therefore:

- **Case 1:** $n \mid (k-1) \implies k-1 = \beta n \implies k \equiv 1 \pmod{n}$.
- **Case 2:** $n \mid (k+1) \implies k+1 = \delta n \implies k \equiv -1 \pmod{n}$, which is equivalent to $k \equiv n-1 \pmod{n}$.

Since k is the remainder modulo n ($0 \leq k < n$), the only possible solutions are $k = 1$ or $k = n-1$. Therefore, $x \equiv 1 \pmod{n}$ or $x \equiv n-1 \pmod{n}$. ■

4.2 The Sequence of Repeated Squaring

This mathematical constraint gave us a powerful new lens. Instead of merely checking if $a^{n-1} \equiv 1 \pmod{n}$ at the very end of our computation, we realized we could inspect the sequence of values generated by repeated squaring $a^s, a^{2s}, a^{4s}, \ldots, a^{2^t \cdot s} \pmod{n}$, where $n-1 = 2^t \cdot s$, and s is an odd integer.

If n is truly prime, Fermat's Little Theorem guarantees the sequence ends in 1. However, our Lemma imposes a much stricter internal condition on this sequence: the value immediately preceding the first appearance of 1 **must** be $n-1$ (which is $-1 \pmod{n}$).

If we encounter a 1 during this sequence, but the preceding value is some $x \notin \{1, n-1\}$, we have uncovered a non-trivial square root of 1 modulo n. By our earlier proofs, this is mathematically impossible for a prime number. Therefore, the existence of such a value acts as an absolute witness to the compositeness of n.

4.3 Final Randomized Algorithm

By formalizing this sequence of checks, we arrived at the following randomized algorithm:

Algorithm 1 Miller Rabin Primality Test

Require: $n > 2$, an odd integer to be tested for primality
Require: k, the number of rounds of testing to perform
Ensure: "composite" if n is found to be composite, "probably prime" otherwise

```
Let s > 0 and d > 0 such that n − 1 = 2^s · d {Factor powers of 2 from n − 1}
for i = 1 to k do
    Choose random a ∈ [2, n − 2]
    x ← a^d mod n
    for r = 1 to s do
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then
            return "composite" {Nontrivial square root of 1 mod n}
        end if
        x ← y
    end for
    if x ≠ 1 then
        return "composite"
    end if
end for
return "probably prime"
```

4.4 Error Analysis

If the algorithm outputs *composite* it is always correct, and if it outputs *probably prime* it is wrong with a probability $\leq \frac{1}{4}$ per iteration. Hence, after k iterations, the probability of falsely identifying a composite number as prime drops exponentially to at most $\left(\frac{1}{4}\right)^k$. The proof of this is highly non-trivial and beyond the scope of this report; however, a detailed proof can be found here.

4.5 Time Complexity Analysis

The total time complexity of the Miller-Rabin test is $\mathcal{O}(k \cdot \log^3 n)$. This is derived from the following operations performed in each of the k iterations:

- **Modular Exponentiation:** The core of the test involves computing $a^d \pmod{n}$. Using the binary exponentiation algorithm (square-and-multiply), this requires $\mathcal{O}(\log d)$ modular multiplications. Since $d < n$, this equates to $\mathcal{O}(\log n)$ multiplications.
- **Multiplication Complexity:** Performing a modular multiplication on numbers with $\log n$ bits typically takes $\mathcal{O}(\log^2 n)$ time using standard algorithms.
- **Total Per Iteration:** Consequently, the cost of a single iteration is $\mathcal{O}(\log n \cdot \log^2 n) = \mathcal{O}(\log^3 n)$.

Across k independent iterations, the cumulative complexity is $\mathcal{O}(k \log^3 n)$. It is worth noting that using advanced fast multiplication techniques can reduce this complexity to $\tilde{\mathcal{O}}(k \log^2 n)$.

4.6 Comparison and Reliability

Unlike Fermat's test, this algorithm leverages deeper arithmetic properties and is not vulnerable to pseudoprimes like Carmichael numbers. The exponential decrease in error probability paired with

its efficiency makes it highly reliable and the preferred choice for practical applications, such as cryptographic RSA key generation.

5 Experimentation and Benchmarking

5.1 AKS Implementation

Figure 3 shows runtime of the AKS primality test on increasing input sizes. Even for small inputs, the runtime grows quickly and hits the 1-second cutoff beyond 4-digit numbers. Despite being polynomial in theory, the constant factors are large enough to make it unusable in practice.

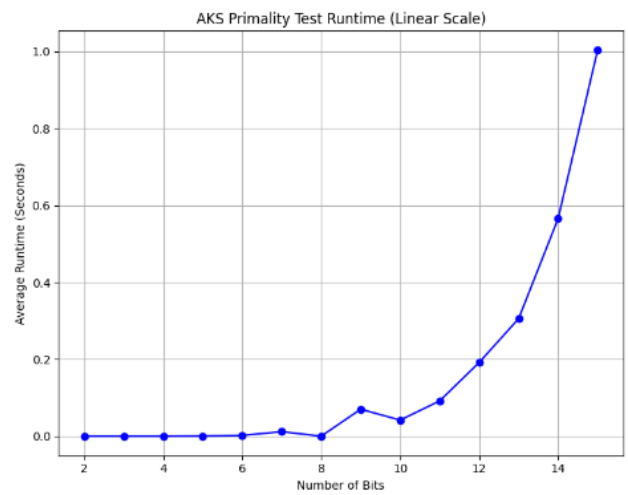

Figure 3: AKS runtime vs Bitsize

5.2 Miller-Rabin Implementation

Figure 4 shows runtime of the Miller-Rabin test ($k = 10$) on inputs up to 4096 bits. The curve stays nearly flat till about 1000 bit numbers and only has 0.25 second runtime for 4000 bit numbers, showing that adding more bits barely increases the time taken. This is what makes Miller-Rabin practical for generating large cryptographic primes.

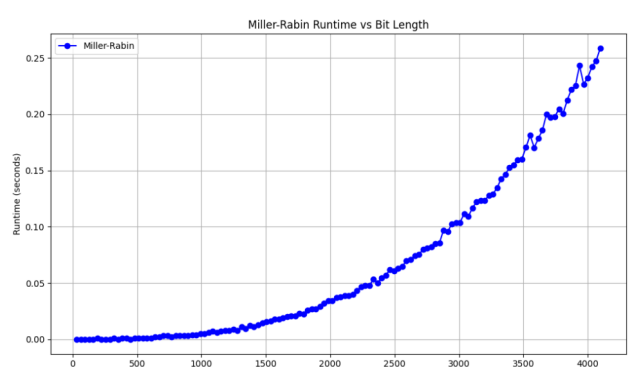

Figure 4: Runtime of the Miller-Rabin test ($k = 10$)

5.3 Analysis

Figure 7 shows runtime comparison between AKS and Miller-Rabin ($k = 10$). In the linear plot (a), AKS shoots up and tends to reach the timeout limit while Miller-Rabin barely registers and has almost a flat graph. The log plot (b) makes it easier to see both curves at once — Miller-Rabin grows slowly and smoothly, while AKS escalates quickly even within the few sizes it manages to finish.

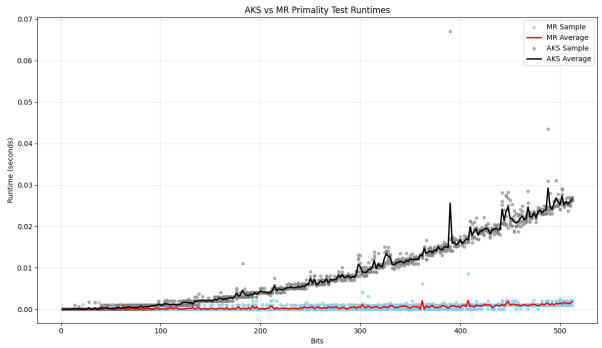

Figure 5: (a) Linear scale

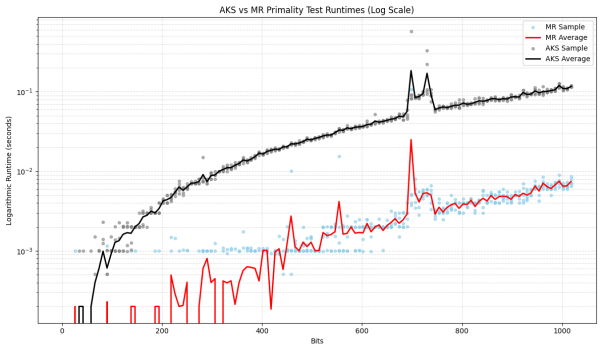

Figure 6: (b) Logarithmic scale

Figure 7: AKS vs Miller-Rabin

Figure 8 shows number of false positives (out of 2000 tests) produced by Miller-Rabin for different values of k, tested on two sets — known Carmichael numbers and randomly generated composites. Random composites are rarely misidentified even at low k, since most composites have many witnesses that expose them. Carmichael numbers, however, cause significantly more false positives at small k but drop to near zero as k increases, showing that more witnesses directly reduces the error probability.

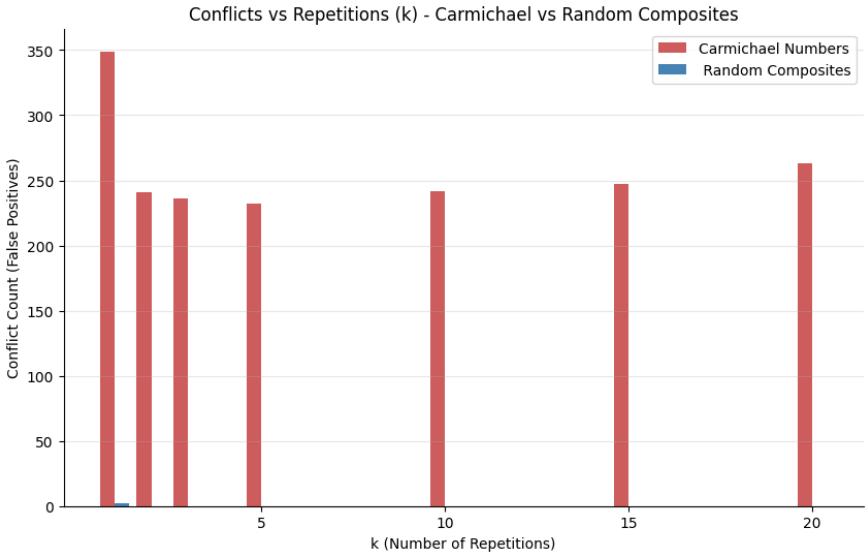

Figure 8: Conflict vs Witness

5.4 Deviation from Average Runtime

To understand how consistent Miller-Rabin is across repeated runs, each input size was benchmarked 2000 times and the spread of runtimes around the mean was recorded. For each input size n, and for each threshold x, we measured what percentage of runs took longer than $x\%$ above the mean. A well-behaved algorithm should see this percentage drop quickly as x increases — meaning most runs stay close to the average and only a few are outliers.

Table 9 shows these percentages across input sizes from 100 to 1000 bits and thresholds from $x = 0$ to $x = 200$. As expected, the $x = 0$ shows large values for all input sizes, since roughly half of all runs should exceed the mean by definition. The values then fall off as x increases, confirming that large deviations are rare.

x	100	200	300	400	500	600	700	800	900	1000
0	31.8	23.8	22.2	23.0	21.4	25.8	26.6	59.8	58.6	24.0
10	23.4	14.4	12.8	10.0	7.0	6.2	17.2	56.2	55.6	5.0
20	13.0	11.8	11.6	9.4	5.8	4.6	13.0	51.0	51.6	3.8
30	11.4	7.2	7.6	8.0	5.6	4.6	7.8	39.6	30.8	3.2
40	10.6	1.6	2.4	1.2	1.0	1.6	4.4	14.4	7.0	0.4
50	7.0	1.0	1.2	0.6	0.2	0.6	2.8	6.4	3.8	0.0
60	6.0	0.8	0.6	0.6	0.2	0.6	2.6	3.8	3.6	0.0
70	5.0	0.4	0.4	0.2	0.2	0.6	2.0	3.0	3.6	0.0
80	5.0	0.2	0.4	0.2	0.0	0.4	2.0	2.4	3.2	0.0
90	4.4	0.2	0.2	0.0	0.0	0.2	1.6	1.4	1.0	0.0
100	4.0	0.2	0.2	0.0	0.0	0.0	1.0	0.6	0.2	0.0
110	3.6	0.0	0.0	0.0	0.0	0.0	1.0	0.4	0.2	0.0
120	2.6	0.0	0.0	0.0	0.0	0.0	0.2	0.4	0.2	0.0
130	2.4	0.0	0.0	0.0	0.0	0.0	0.2	0.4	0.2	0.0
140	2.2	0.0	0.0	0.0	0.0	0.0	0.2	0.2	0.2	0.0
150	1.6	0.0	0.0	0.0	0.0	0.0	0.2	0.2	0.0	0.0
160	1.0	0.0	0.0	0.0	0.0	0.0	0.2	0.0	0.0	0.0
170	0.6	0.0	0.0	0.0	0.0	0.0	0.2	0.0	0.0	0.0
180	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
190	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
200	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Figure 9: Percentage of runs exceeding the mean runtime by at least $x\%$, for input sizes 100–1000 bits. Values decrease as x increases, showing that most runs stay close to the average.

Figure 10 breaks this down per input size. For smaller inputs like 100–300 bits, the curve drops slowly, meaning there is more relative spread. For larger inputs, the runtimes are longer and more stable, so the curve drops sharply and the tail is short.

Figure 11 overlays all input sizes on a single plot for easier comparison. The curves for larger bit sizes drop off faster and reach near-zero earlier, showing that Miller-Rabin becomes more consistent and predictable as input size grows. This is consistent with the $\mathcal{O}(k \cdot \log^2 n)$ complexity — at larger inputs the actual computation dominates and random system noise contributes less to the total runtime.

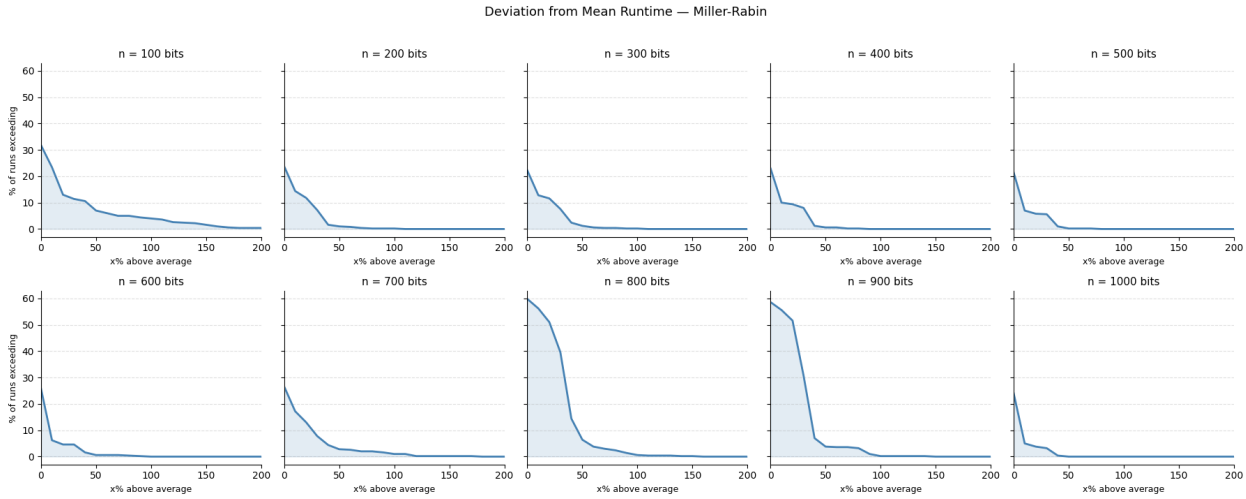

Figure 10: Deviation curves for each input size individually. Each plot shows the percentage of runs exceeding the mean by at least $x\%$. Smaller inputs show a slower dropoff due to high relative variance from timer resolution noise; larger inputs converge faster.

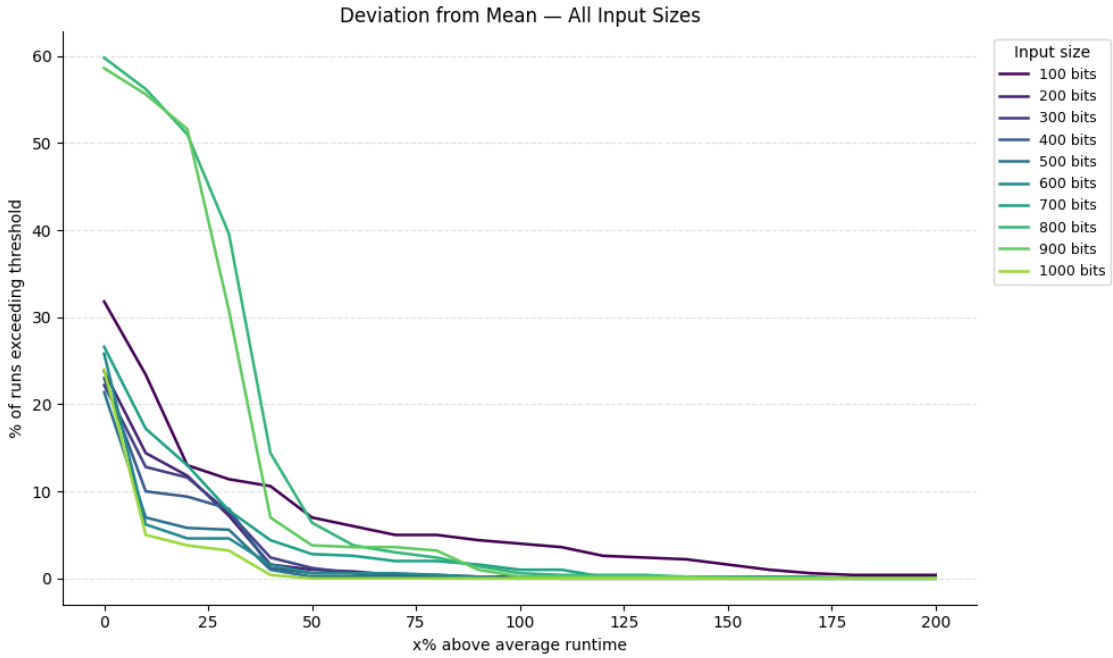

Figure 11: Deviation curves for all input sizes overlaid. Larger inputs (darker lines) drop off faster, indicating more consistent runtimes. Smaller inputs have longer tails due to the algorithm finishing faster than the timer resolution.

6 Interactive Exploration and Analysis of Miller-Rabin vs. AKS

To empirically validate the theoretical performance of primality testing algorithms, an interactive visualization tool was developed to compare the Miller-Rabin and Agrawal-Kayal-Saxena (AKS) primality tests. The explorer evaluates the algorithms across four primary dimensions: computational race simulations, error budget analysis, practical application in RSA key generation, and empirical time complexity surfaces.

6.1 Asymptotic vs. Empirical Complexity

Theoretical analysis establishes that the AKS algorithm runs in deterministic polynomial time, specifically $\mathcal{O}(\log^6 n)$, while the probabilistic Miller-Rabin test operates in $\mathcal{O}(k \cdot \log^2 n)$, where k is the number of testing rounds. However, the interactive simulator demonstrates that the hidden constant in the AKS time complexity is astronomically large.

Empirical surface plots within the explorer reveal that the execution time for Miller-Rabin remains practically constant (well under 1 ms) across evaluated bit-lengths and k-values. Conversely, the AKS execution time rises sharply as a function of bit-length, dominated by the polynomial ring arithmetic required in its algorithmic execution. The operational crossover point where AKS outperforms Miller-Rabin lies at numbers with tens of thousands of digits—far beyond the constraints of standard cryptographic parameters.

Figure 12: *
(a) Operation count race between Miller-Rabin and AKS on a 512-bit input with $k = 10$. AKS requires 474,000× more operations despite being polynomial in theory.

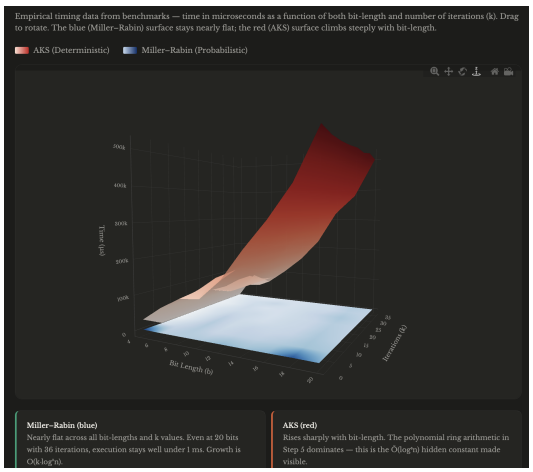

Figure 13: *
(b) 3D surface plot of execution time vs. bit-length and k. Miller-Rabin (blue) stays flat; AKS (red) climbs steeply, making the $\tilde{\mathcal{O}}(\log^6 n)$ hidden constant visible.

Figure 14: Empirical complexity comparison between Miller-Rabin and AKS across input sizes and iteration counts.

6.2 Probabilistic Error Bounds and Hardware Realities

A common critique of the Miller-Rabin algorithm is its probabilistic nature. The explorer contextualizes the false-positive probability, defined bounded by 4^{-k}. The visualization charts this error decay logarithmically, comparing algorithmic error rates to real-world physical events.

The simulator highlights that at $k \leq 3$, the error rate is statistically significant. However, by $k = 15$, the probability of a false positive drops below the baseline probability of a cosmic ray inducing a random bit-flip in physical RAM during the computation (10^{-12}). At the standard cryptographic parameter of $k = 20$, the error rate approaches theoretical zero. Consequently, the practical distinction between a "probabilistic" result at $k = 20$ and a "deterministic" result is negligible in applied cryptography.

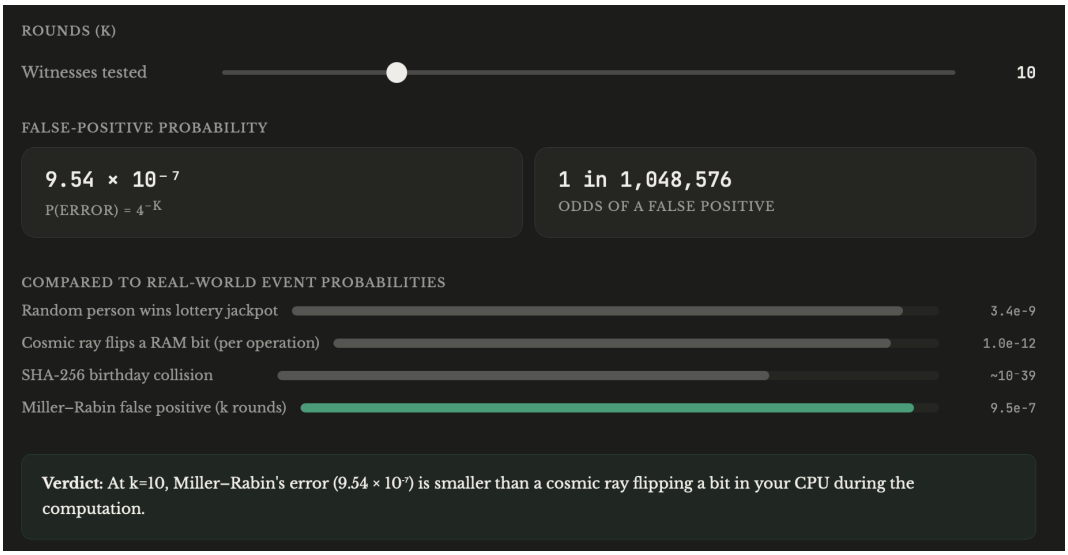

Figure 10: False-positive probability of Miller-Rabin at $k = 10$, shown as 9.54×10^{-7} (1 in 1,048,576), compared against real-world event probabilities. Notably, the error rate at $k = 10$ is already higher than a cosmic ray flipping a RAM bit (10^{-12}) or a SHA-256 birthday collision ($\sim 10^{-39}$), but increasing k further closes this gap rapidly. This contextualises why $k = 40$ is considered cryptographically safe.

6.3 Practical Application: RSA Key Generation

The performance gap is most evident when applied to the generation of large primes for RSA cryptosystems. Generating an RSA-1024 key requires finding two prime factors, p and q, each approximately 512 bits in length. According to the Prime Number Theorem, the density of primes at this magnitude dictates that roughly 1 in every 355 candidates will be prime.

The explorer's RSA simulation emphasizes the necessity of testing thousands of candidates. Because both algorithms must process the exact same sequence of candidates to locate a prime, the per-test cost dictates the overall generation time. The simulation confirms that Miller-Rabin can generate an RSA-1024 keypair in milliseconds, processing tens of thousands of tests per second. In contrast, running the deterministic AKS algorithm for the same task results in unfeasible execution times, yielding a computational speedup of several orders of magnitude in favor of Miller-Rabin.

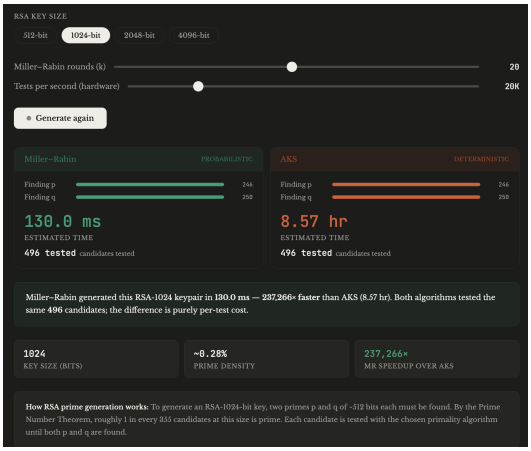

Figure 11: *
(a) Simulated RSA-1024 generation: Miller-Rabin completes in 130 ms vs. 8.57 hours for AKS, testing the same 496 candidates — a 237,000× speedup.

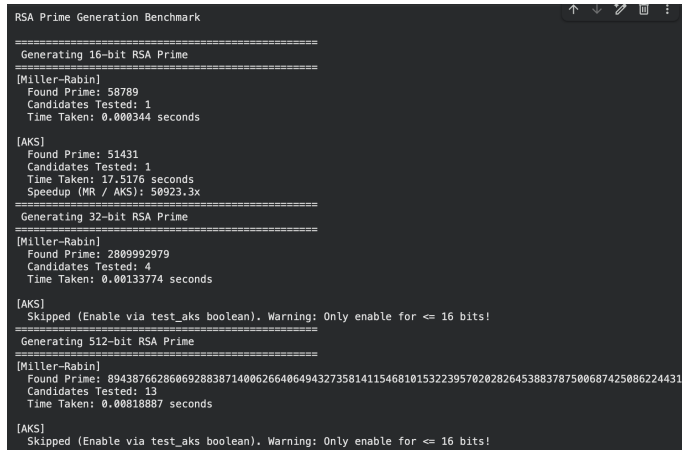

Figure 12: *
(b) Benchmark output at 16-bit and 32-bit sizes. At 16 bits, Miller-Rabin is 50,923× faster. AKS is skipped entirely beyond 16 bits due to impractical runtime.

Figure 13: RSA prime generation performance: interactive simulation and empirical benchmark results.

7 Conclusion

Through this comprehensive study and experimental analysis, we have explored the critical role of randomized algorithms in the field of primality testing. While the **AKS algorithm** represents a significant theoretical milestone as the first deterministic, polynomial-time test, our empirical results confirm that it remains **computationally impractical** for real-world applications due to its high constant factors and $\mathcal{O}(\log^6 n)$ complexity.

In contrast, randomized methods, specifically the **Miller-Rabin test**, demonstrate clear superiority in practical environments:

- **Efficiency:** Miller-Rabin operates with an average time complexity of $\mathcal{O}(k \log^2 n)$ (or $\mathcal{O}(k \log^3 n)$ with standard multiplication), allowing it to generate 1024-bit RSA keys in milliseconds, whereas AKS would require several hours to process the same candidates.

- **Reliability:** Although probabilistic, the Miller-Rabin test is not vulnerable to Carmichael numbers, unlike the Fermat test. By increasing the number of iterations k, the probability of error decreases exponentially ($\leq 4^{-k}$).

- **Practical Determinism:** At $k = 20$ or $k = 40$, the error probability drops below the rate of hardware-level failures, such as cosmic-ray induced bit-flips, rendering the algorithm *effectively deterministic* for cryptographic purposes.

Ultimately, randomized algorithms remain the backbone of modern secure communication. Their ability to provide a tunable trade-off between runtime and accuracy makes them indispensable for the generation of large primes in RSA and Diffie-Hellman key exchanges.

8 Acknowledgment

We express our gratitude to **Professor Surender Baswana** for his unwavering guidance and support throughout the duration of this project. His assistance significantly influenced the course of our journey, making it markedly distinct from what it would have been otherwise.

All code used can be found at the below repository:
`https://github.com/unordinarysubstance/Primality_Testing_CS648`

References

[1] Keith Conrad. *Carmichael numbers and Korselt's criterion*. 2004. Available online at: `https://kconrad.math.uconn.edu/blurbs/ugradnumthy/carmichaelkorselt.pdf`.